

Introduction to DL and GenAI Project

Author: Priyam Tiwari | Roll Number: 24f2005398

1. Executive Summary

This project solves the Smart MCQ Solver Challenge on Kaggle, where each question has a prompt and 5 answer options (A–E), and the goal is to predict the top-3 most likely correct answers in ranked order. Performance is measured using Mean Average Precision at 3 (MAP@3), which rewards a model more for ranking the correct answer earlier in its top-3 list.

Three different model types were built and compared, along with a simple untrained baseline for reference:

Model Type	MAP@3	Notes
Baseline: TF-IDF + Cosine Similarity	0.3119 (train)	Unsupervised, no training — reference lower bound
BiLSTM + Attention (From Scratch)	0.71238 (Kaggle)	Custom RNN, no pretrained weights — built entirely from scratch
TF-IDF + Logistic Regression	0.73815 (Kaggle)	Classical ML — fast, interpretable
ELECTRA-base-discriminator	0.75519 (Kaggle)	Fine-tuned pretrained transformer — best score

Key Findings: ELECTRA gave the best score by using pretrained language knowledge. Logistic Regression, despite being far simpler and needing no GPU, also crossed the competition's qualifying cutoff of 0.73. The from-scratch model performed respectably without any pretrained knowledge. All three trained models clearly beat the untrained baseline, showing they learn real patterns rather than just matching keywords.

Kaggle Competition Performance: Best submission (ELECTRA-base-discriminator) — MAP@3 = 0.75519.

2. Introduction

Problem Statement

The Kaggle competition requires building a system that, given a question and 5 options, predicts the top 3 most likely correct answers in ranked order. The system must handle questions from diverse topics (science, philosophy, general knowledge) and options that are often worded similarly to each other, making the task harder than simple keyword matching.

Project Objective

- Build and compare three different models: one from scratch, one pretrained, and one additional model of choice.
- Track every experiment using Weights & Biases for fair comparison.
- Cross the competition's qualifying MAP@3 cutoff of 0.73.
- Deploy the working system as a usable, interactive application.

3. Dataset & Preprocessing

The dataset has three files: train.csv (~2,000 labeled questions), test.csv (unlabeled, for submission), and sample_submission.csv. Each row has an id, a prompt, five options (A–E), and, in train.csv, the correct answer. Questions cover topics like physics, philosophy, and biology, and appear to be synthetically generated.

Exploratory Data Analysis

- No missing values found in the dataset.
- A few duplicate prompts were checked and consistently had the same correct answer.
- Correct-answer distribution is mildly imbalanced (16%–24.5% across A–E), not severe.
- Average prompt length is ~17–19 words, with no link between length and the correct answer.
- Vocabulary size was measured to help size the embedding layer for the from-scratch model.

Data Preprocessing

All text was lowercased and had punctuation removed using a simple cleaning function. No missing-value handling was needed. Tokenization differs per model (see Section 4).

Data Augmentation

Not used. The available data was enough for two of the three models to cross the qualifying cutoff, so augmentation was left as future work.

4. Tokenization Strategy

The best model (ELECTRA-base-discriminator) uses its associated WordPiece tokenizer, loaded via Hugging Face's AutoTokenizer. WordPiece splits rare words into sub-word units (e.g., “unhappiness” → “un” + “##happiness”), so it can represent a huge effective vocabulary with a fixed set of ~30,000 tokens. This suits the dataset well since questions span many domains with specialized words, and it lets ELECTRA reuse its pretrained knowledge during fine-tuning.

Each question was formatted as 5 (prompt, option) pairs, tokenized with max length 128, padding to max length, and truncation enabled, in batches of 16.

Two other tokenization approaches were used and compared: a custom regex tokenizer for the from-scratch model (vocabulary built only from training data, no pretrained knowledge), and a TF-IDF vectorizer for Logistic Regression (weights word/phrase frequency directly instead of producing token sequences). WordPiece + a pretrained transformer worked best overall, since it combines flexible sub-word handling with transferred language knowledge the other two approaches lack.

5. Modeling & Experimentation

Three models were built: one fine-tuned pretrained model, one from-scratch deep learning model, and one additional model of choice. The from-scratch model is itself RNN-based (BiLSTM), so it also covers the “RNN-based model” category; the third slot is a classical ML model (Logistic Regression).

Model 1. TF-IDF + Logistic Regression

The prompt and all 5 options are combined into one text per question, then vectorized with TF-IDF (unigrams to trigrams, 100,000 features, English stop-words removed). A multinomial Logistic Regression classifier ($C=10.0$) predicts the correct option, and the top-3 by probability form the prediction.

Training Strategy:

- Solver: lbfgs, max_iter = 2000
- Regularization strength $C = 10.0$
- TF-IDF: ngram_range=(1,3), sublinear_tf=True

Model 2. BiLSTM + Attention (Built From Scratch)

Built entirely from scratch — no pretrained weights anywhere. A custom tokenizer and vocabulary are built from the training data. Word embeddings (size 128) are randomly initialized. A shared BiLSTM (hidden size 128) encodes the prompt and each option. An attention layer lets each option focus on the most relevant prompt words. The option vector, attention context, their difference, and their product are combined and passed through a small MLP to score each option.

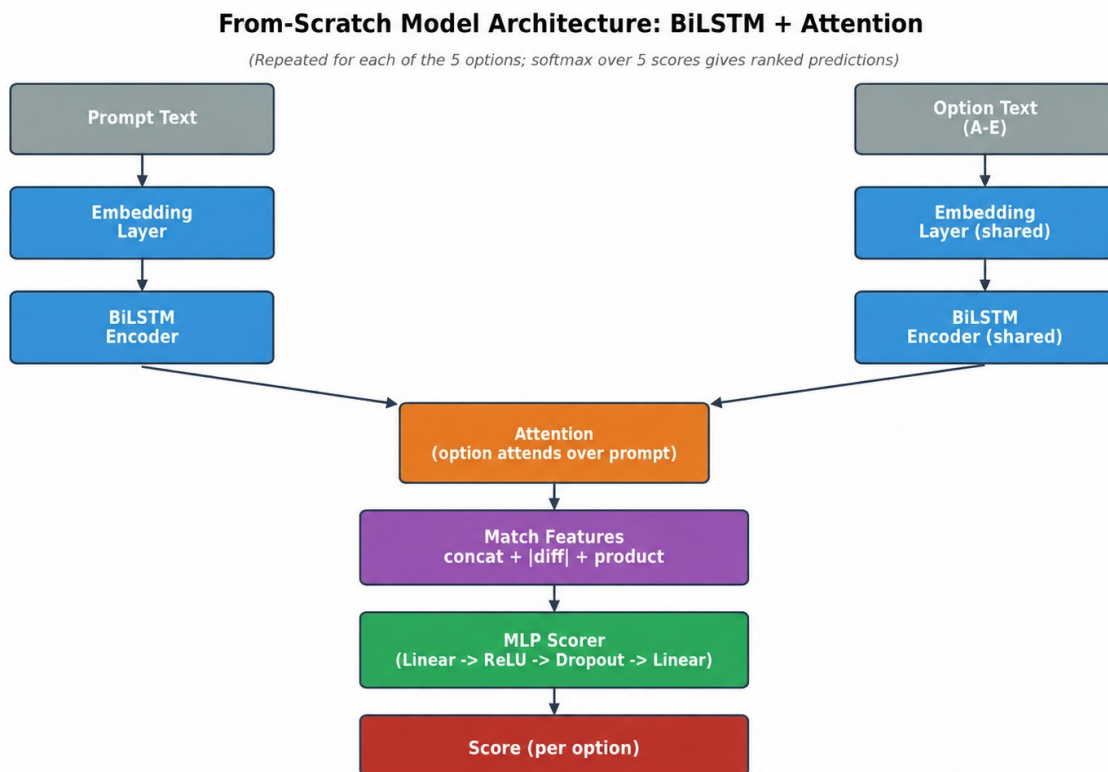


Figure 1: BiLSTM + Attention architecture.

Layer details:

- Embedding layer (learned from scratch, size 128)
- Bidirectional LSTM encoder (captures context from both directions)
- Attention (lets each option focus on relevant prompt words)
- Dropout (0.3) for regularization

Training Strategy:

- Optimizer Adam ($\text{lr}=1\text{e-}3$)
- CrossEntropyLoss
- 15 epochs, batch size 32, gradient clipping at 5.0
- 85/15 train-validation split; best checkpoint by validation MAP@3 kept

Model 3. ELECTRA-base-discriminator (Pretrained, Fine-Tuned)

ELECTRA (Clark et al., 2020) is pretrained using a replaced-token-detection task instead of masked-language-modeling like BERT, making it sample-efficient. This project fine-tunes google/electra-base-discriminator using Hugging Face's AutoModelForMultipleChoice, treating each question as a 5-way classification problem.

Training Strategy:

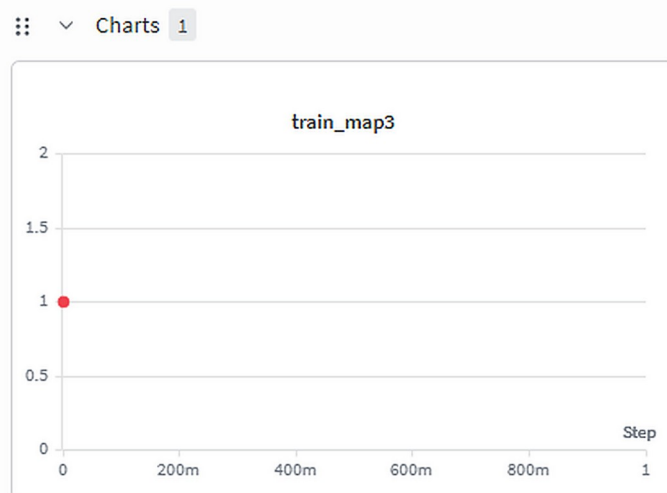
- Optimizer AdamW ($\text{lr}=3\text{e-}5$, $\text{weight_decay}=0.01$)
- Linear scheduler with warmup (10% of total steps)
- 3 epochs, batch size 16, gradient clipping at 1.0
- Stratified 90/10 train-validation split

6. Performance & Comparative Analysis

MAP@3 is used as the primary evaluation metric for the Kaggle competition. Accuracy and macro F1-score were also tracked for the from-scratch model to monitor per-class performance given the dataset's mild class imbalance.

Training Performance

Model 1. TF-IDF + Logistic Regression



W&B tracking: model1-tfidf-logistic (train_map3)

Track Performance Graph of TF-IDF Logistic Regression: <https://wandb.ai/priyamtiwari948-indian-institute-of-technology-madras/smart-mcq-solver/runs/1xffwqz2?nw=nwuserpriyamtiwari948>

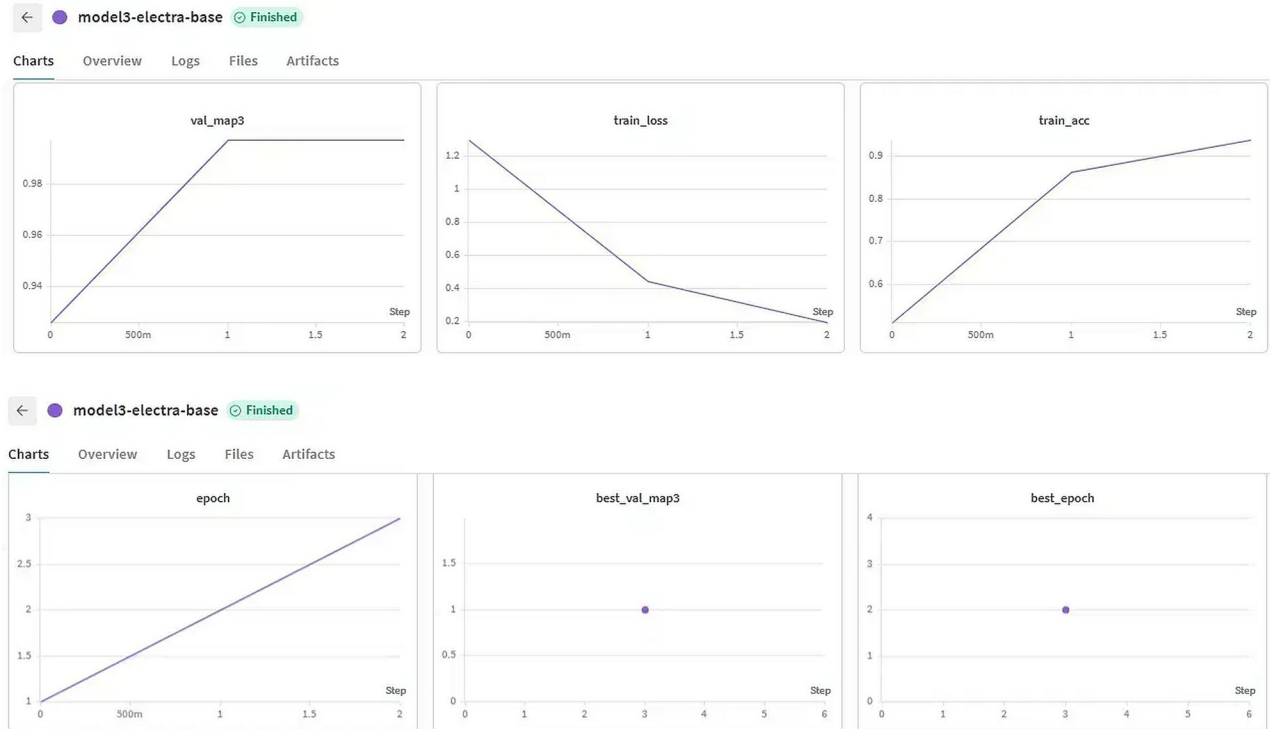
Model 2. BiLSTM + Attention (From Scratch)



W&B tracking: model2-scratch-bilstm-attn (val_map3, val_loss, val_f1, val_acc, train_map3, train_loss, train_f1, train_acc, epoch)

Track Performance Graph of Scratch BiLSTM Attention: <https://wandb.ai/priyamtiwari948-indian-institute-of-technology-madras/smart-mcq-solver/runs/wuxnzgyf?nw=nwuserpriyamtiwari948>

Model 3. ELECTRA-base-discriminator



W&B tracking: model3-electra-base (val_map3, train_loss, train_acc, epoch, best_val_map3, best_epoch)

Track Performance Graph of ELECTRA-base-discriminator: <https://wandb.ai/priyamtiwari948-indian-institute-of-technology-madras/smart-mcq-solver/runs/1wb253bn?nw=nwuserpriyamtiwari948>

Comparative Report

Model	Category	Evaluation	MAP@3
Baseline: TF-IDF + Cosine Similarity	Unsupervised baseline	Train (local)	0.3119
BiLSTM + Attention (From Scratch)	Custom / RNN-based	Kaggle leaderboard	0.71238
TF-IDF + Logistic Regression	Classical ML	Kaggle leaderboard	0.73815
ELECTRA-base-discriminator	Pretrained transformer	Kaggle leaderboard	0.75519

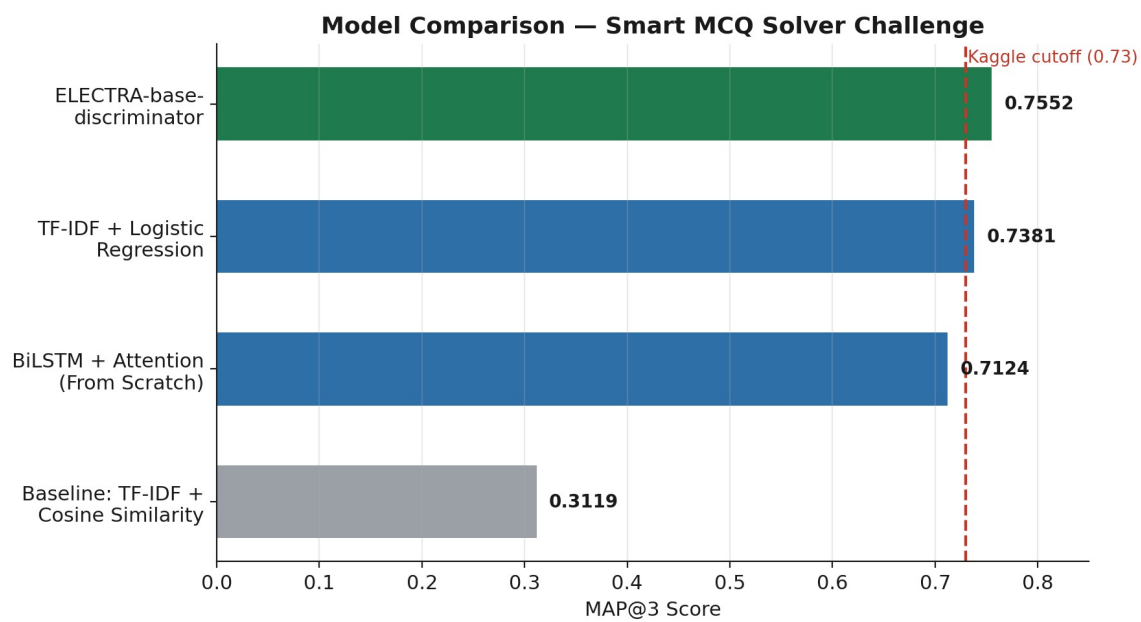


Figure 2: MAP@3 comparison, with the Kaggle cutoff (0.73) marked.

Discussion: ELECTRA scored highest but needed a GPU and the longest training time. Logistic Regression trained in under two minutes on CPU and still crossed the cutoff. The from-scratch model needed the most design effort and a GPU, yet scored slightly below Logistic Regression — reasonable, since it learns everything from a small dataset with no pretrained knowledge. All three clearly beat the untrained baseline (~0.31), showing they learn real patterns, not just keyword overlap.

Kaggle Performance: Final leaderboard scores — ELECTRA: 0.75519, Logistic Regression: 0.73815, From-scratch model: 0.71238. ELECTRA and Logistic Regression both cross the qualifying cutoff of 0.73.

7. Conclusion & Future Work

Key Learnings

- Fine-tuning a pretrained transformer gives the strongest results with the least manual design effort, but needs GPU compute.

- A well-built classical ML pipeline can be surprisingly competitive, training in a fraction of the time.
- Building a model from scratch deepened understanding of how embeddings, BiLSTM, and attention each contribute to a matching task.
- Tracking every model on Weights & Biases made fair comparison much easier.

Challenges Faced

- Keeping the untrained baseline separate from trained models, to fairly credit real learning.
- Avoiding accidental reuse of weights across repeated ELECTRA training runs, which once gave misleadingly perfect validation scores.
- Finding that a bigger from-scratch model with more epochs actually overfit and lowered the leaderboard score, so the original smaller setup was kept.

Areas for Improvement

- Ensemble predictions across all three models to potentially raise MAP@3 further.
- Use data augmentation (synonym replacement, option shuffling) to help the from-scratch model.
- Try larger pretrained transformers and tune hyperparameters using W&B sweeps.

8. Model Deployment

The from-scratch BiLSTM + Attention model is deployed as an interactive web app using Gradio on Hugging Face Spaces (ZeroGPU hardware). Users can enter any question and 5 options and get real-time top-3 ranked predictions with confidence scores.

Live demo: huggingface.co/spaces/priyamtiwari948/smart-mcq-solver

9. Key References

- Smart MCQ Solver Challenge – Kaggle competition page and data description.
- Google ELECTRA-base discriminator – original paper and Hugging Face model card.
- PyTorch documentation – GRU, Embedding, Dropout, BatchNorm, optimizers, schedulers, loss functions.
- Hugging Face Transformers – multiple-choice models and tokenizers.
- Scikit-learn – TfidfVectorizer, LogisticRegression, evaluation metrics.
- Weights & Biases – experiment tracking and visualization tools.